RestaurantRush - VIP C2 during Happy Hour (1/3)

Manager
cli:RestaurantCLI
gc:GameController
tm:TurnManager
arr:ArrivalSchedule
hh:HappyHour
r:Restaurant
host:Host
waiter:Waiter
chef:Chef
cashier:Cashier
t2:Table

Turn 4 - manager choice, arrival, order and READY

Turns 1-3 complete; Happy Hour unused; table 2 free.

1. Happy Hour> on
2. startTurn(true)
3. beginTurn(r, true)
4. arrivalFor(4)
5. new VIPCustomer(2)
c2:VIPCustomer
6. c2 {WAITING, satisfaction 100}
7. c2
8. addArrival(c2)
opt **[activateHappyHour == true]**
9. activate(4)
10. Host> seat 2 2
11. seatCustomerWithHost(2, 2)
12. seat(r, 2, 2)
13. seat(2, 2)
14. assign(c2)
15. seat() {SEATED}
16. Waiter> order 2
17. takeOrder(2)
18. takeOrder(r, 2, 4)
19. chooseMenuItem(r.getMenu())
20. burger {1500 cents, 1 prep unit}
21. quotePriceCents(burger)
22. 1350 cents {VIP discount}
23. quotePriceCents(1350, 4)
24. 1080 cents
Manager choice changes the bill:
without Happy Hour $13.50; chosen ON $10.80.
25. new Order(2, burger, 1080)
o2:Order
26. o2 {PLACED; bill locked}
27. registerOrder(2, o2)
28. acceptOrder(o2)
29. Chef> cook 2
30. prepareOrder(2)
31. prepareOneUnit(o2)
32. prepareOneUnit()
[canPrepare()]
1 -> 0 units; READY
33. ActionResult.ok("Burger is READY to serve.")
34. finishTurn()
35. resolveTurn(r)
36. applyWaitingLoss() {100 -> 95}
37. advanceTurn()

RestaurantRush - C2 service and eating (2/3)

Manager
cli:RestaurantCLI
gc:GameController
tm:TurnManager
arr:ArrivalSchedule
hh:HappyHour
r:Restaurant
host:Host
waiter:Waiter
chef:Chef
cashier:Cashier
t2:Table

c2:VIPCustomer (continued)
o2:Order (continued)

Turn 5 - recover satisfaction and serve the READY order

38. startTurn(false)
39. beginTurn(r, false)
opt **[Happy Hour active on turn 5]**
40. recoverWaitingSatisfaction(15)
{95 -> 100}
41. Host> wait
42. waitHost()
43. Waiter> serve 2
44. serveOrder(2)
45. serve(r, 2, 5)
46. serve(2, 5)
[C2 SEATED and registered Order READY]
47. markServed() {SERVED}
48. markServed(5)
{EATING; servedTurn = 5}
49. Chef> wait
50. waitChef()
51. finishTurn()
52. resolveTurn(r)
53. finishEating(5)
[5 < 5] false:
stays EATING
54. advanceTurn()

Turn 6 - one full turn of eating completes

Turn 6 starts; Host, Waiter and Chef all wait.
55. finishTurn()
56. resolveTurn(r)
loop **[each active customer (C2 shown)]**
57. finishEating(6)
[EATING and servedTurn 5 < 6]
READY_TO_PAY; readyToPayTurn = 6
58. collectOne(r, 6)
59. oldestEligiblePayer(6)
60. null {[readyToPayTurn < 6]
false}
61. false {no payment}
62. advanceTurn()

RestaurantRush - C2 payment and final result (3/3)

Manager
cli:RestaurantCLI
gc:GameController
tm:TurnManager
arr:ArrivalSchedule
hh:HappyHour
r:Restaurant
host:Host
waiter:Waiter
chef:Chef
cashier:Cashier
t2:Table
c2:VIPCustomer (continued)
o2:Order (continued)

Turn 7 - automatic payment and table release

Turn 7 starts; C3 arrives. Host seats C3,
Waiter takes C3's order, Chef cooks. All choices complete.

63. finishTurn()
64. resolveTurn(r)
65. collectOne(r, 7)
66. oldestEligiblePayer(7)
67. c2 {[readyToPayTurn 6 < 7] true}
68. recordPayment(c2, 7)
69. markPaid() {PAID}
70. markPaid() {PAID}
71. release() {table 2 free}

Add the locked 1080-cent bill to revenue;
record C2's paid satisfaction of 100.

72. true
73. true {one payment this turn}
74. advanceTurn()

Turns 8-18 - continue the game

loop [remaining turns while !controller.isFinished()]

75. startTurn(false)

ref Host, Waiter, Chef choices in the same order as above

76. finishTurn() {resolve automatic events, then advance}

After turn 18 - result check

77. getSnapshot()
78. snapshot {5 paid; $71.10; average 84.8}
79. isVictory()

[paid >= 4 AND revenue >= 4500 cents
AND average paid satisfaction >= 60]

80. victory = true
81. displayOutcome(snapshot, victory)

alt [victory == true (chosen execution)]
82. VICTORY and final statistics
[victory == false]
83. DEFEAT and final statistics